SISTEMAS DISTRIBUIDOS

LABORATORIO S08

Que la transferencia no deje dinero en el aire

Fecha de entrega: 16 de octubre, 2026
Profesor: Ing. Oscar Manual Ruiz Hurtado

Alumno: Echeverría Goicochea Erick Isaac
Equipo: DiDistribuidores

No. Cuenta: 424153520
Grupo: 2
Semestre: 2027-1

Bitácora Lab S08 · Que la transferencia no deje dinero en el aire

Sistemas Distribuidos 2027-1 · Grupo 2 · S08

Alumno: Echeverria Goicochea Erick Isaac

1. El problema (Parte 0) [7:20 PM]

Qué hice: Creé dos cuentas por el gateway: origen S08O1792027643 con saldo 1000 y destino S08D1792027715 con saldo 500. Levanté transferencia con una pausa de 20 segundos entre el cargo y el abono (`PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000`). Lancé una transferencia de 200 del origen al destino y, durante la pausa, apagué cuenta con docker compose stop cuenta. Después volví a encender cuenta y consulté los saldos.

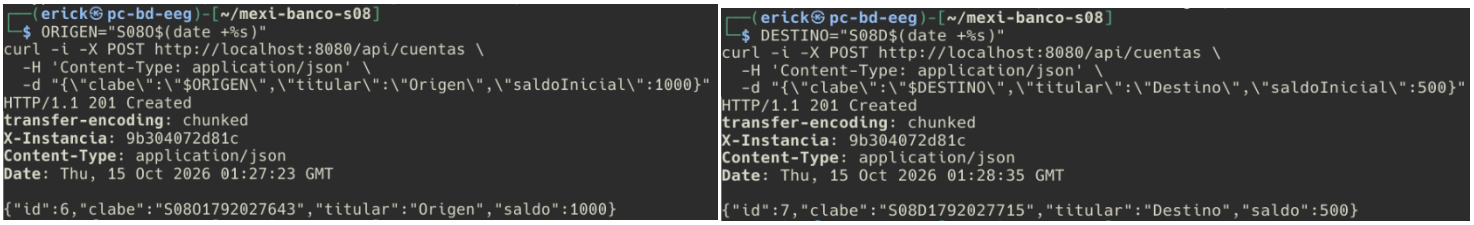


Ilustración 1. Creación de dos cuentas con saldo de 1000 y 500 respectivamente.

Momento	Saldo origen	Saldo destino
Antes de la transferencia	1000	500
Después (con cuenta apagada en la pausa)	800	500

Levanta transferencia con una pausa entre el cargo y el abono: PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia.



Ilustración 2. Transferencia con pausa entre cargo y abono

Lanza una transferencia y, durante esos 20 segundos, apaga cuenta (docker compose stop cuenta).



Ilustración 3. Lanzamiento de transferencia y se apaga cuenta (7:38:19 PM)

Salida:

HTTP/1.1 504 Gateway Timeout

Content-Type: application/json

Content-Length: 140

{"timestamp":"2026-10-15T01:38:29.174+00:00","path":"/api/transferencias","status":504,"error":

"Gateway Timeout","requestId":"adbebfc1-326"}

- Saldos antes y después

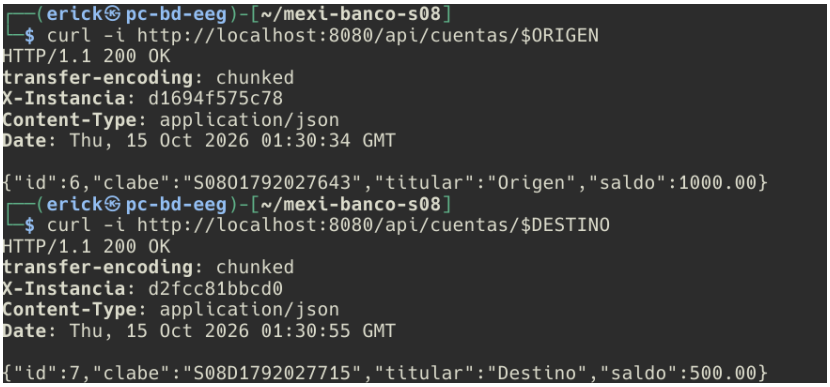


Ilustración 4. Saldos antes de la transferencia

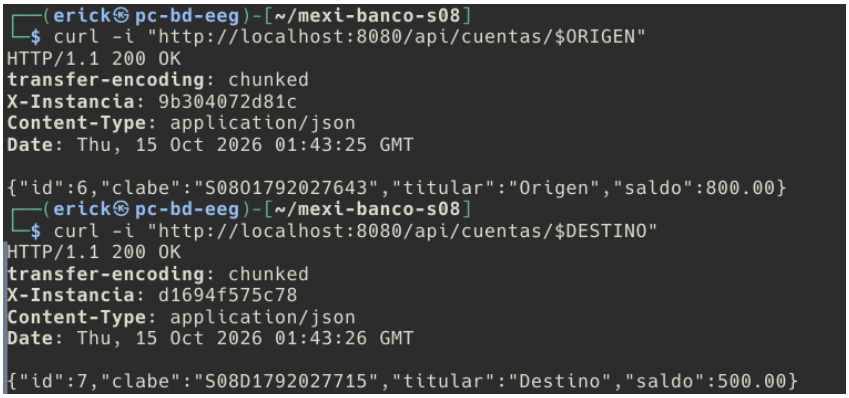


Ilustración 5. Saldos después

Qué respondió la transferencia. El cliente recibió 504 Gateway Timeout. Esto pasó porque el gateway espera máximo 15 segundos (response-timeout: 15s) y la pausa era de 20. El cliente no se enteró de qué pasó con su dinero.

Por qué pasó, en una frase. Transferir ya no es una sola operación: el cargo y el abono son dos peticiones HTTP separadas a otro servicio con su propia base, y cada una se confirma por su cuenta, así que ya no hay un @Transactional que las junte. Como el cargo ya estaba confirmado y el abono nunca llegó, salieron 200 del origen y no llegaron a ningún lado.

Mi saga

Convertí la transferencia en una **saga orquestada**. El orquestador es TransferenciaService: él decide qué paso sigue y qué hace si uno falla. Después de cada paso guarda el estado en la tabla de transferencias, así que si el servicio se cae a la mitad, la base dice en qué paso se quedó.

Pasos y compensación de cada uno

Paso	Estado que queda	Si falla...
Se recibe la solicitud	INICIADA	Todavía no se movió dinero.
Cargo al origen	CARGADA	Estado FALLIDA. No salió dinero, no hay nada que deshacer.
Abono al destino	ABONADA	Compensar: se abona el monto de vuelta al origen y queda COMPENSADA.
Se registran los dos movimientos	COMPLETADA	No se compensa, porque el dinero ya llegó bien. El estado se queda en ABONADA y dice hasta dónde se llegó.
Avisos al cliente	(ya está COMPLETADA)	Solo se anota en el log. Un aviso no se puede "desenviar", por eso va al final.

Diagrama de estados

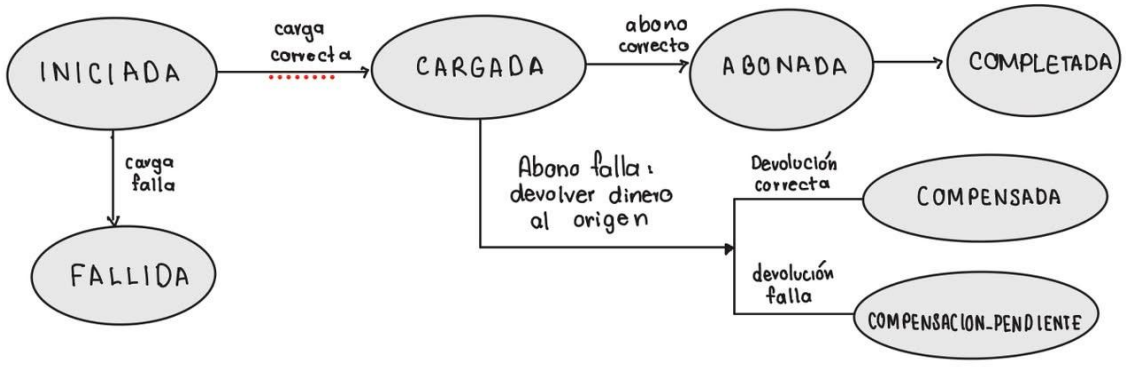


Ilustración 6. Diagrama de estados

Por qué es una saga orquestada

El flujo completo está escrito en un solo lugar (transferir()), que es la ventaja de la orquestación: es fácil de leer, probar y depurar. La desventaja es que el orquestador debe guardar su estado para sobrevivir caídas, y por eso cada paso se guarda en la base.

Código clave [8:00 PM]:

1. **Estado en la entidad.**

   Agregué a Transferencia un enum EstadoTransferencia, los campos estado y detalle, y un método marcar(...) para cambiarlos. La columna quedó sin "obligatoria" a propósito, porque la tabla ya tenía filas de la práctica anterior (ids 1 y 2) y la base no deja agregar una columna obligatoria a una tabla con datos.

2. **Guardar antes de mover dinero.**

   En el código original la transferencia se guardaba hasta el final, así que si el servicio se caía a la mitad no quedaba ningún registro. Ahora se guarda al principio con estado INICIADA y después de cada paso se actualiza. Cada save es su propia transacción local.

```
Transferencia t = transferencias.save(new Transferencia(claveOrigen,
claveDestino, monto)); // INICIADA

CuentaRemota origen;
try {
    origen = cuentas.cargar(claveOrigen, monto);
} catch (RuntimeException e) {
    guardar(t, EstadoTransferencia.FALLIDA, "No se pudo cargar: " +
e.getMessage());
    throw e;
}
guardar(t, EstadoTransferencia.CARGADA, null);
```

3. **Compensación.**

   Si el abono falla, el método compensar(...) le devuelve el monto a la cuenta origen con un abono, marca COMPENSADA y registra un movimiento con la referencia "compensación de la transferencia #id". Después responde 409 explicando qué pasó, en vez de un 500 sin explicación. Registré la devolución con el tipo TRANSFERENCIA_RECIBIDA porque el servicio movimiento no tiene un tipo de compensación y en esta práctica solo entrego transferencia.

```java
private void compensar(Transferencia t, RuntimeException causa) {
    // Idempotencia por estado: solo se devuelve si el dinero sigue "en el aire"
    if (t.getEstado() != EstadoTransferencia.CARGADA
            && t.getEstado() != EstadoTransferencia.COMPENSACION_PENDIENTE) {
        return;
    }
    try {
        CuentaRemota devuelta = cuentas.abonar(t.getClaveOrigen(), t.getMonto());
        guardar(t, EstadoTransferencia.COMPENSADA, "Falló el abono (" +
causa.getMessage() + "); se devolvió el dinero");
        registrarDevolucion(t, devuelta);
    } catch (RuntimeException fallaLaDevolucion) {
        guardar(t, EstadoTransferencia.COMPENSACION_PENDIENTE,
            "Falló el abono y también la devolución: " + fallaLaDevolucion.getMessage());
        throw new ConflictoException("Transferencia " + t.getId() + " en estado
COMPENSACION_PENDIENTE: ...");
    }
    throw new ConflictoException("Transferencia " + t.getId() + " en estado COMPENSADA:
...");
}
```

El estado pasa a COMPENSADA inmediatamente después de que la devolución funciona, y antes de registrar el movimiento. Así, si el movimiento fallara, un reintento no devolvería el dinero dos veces.

4. **Falla provocada del abono.**

Para probar la compensación sin apagar nada agregué la variable de entorno FALLAR_ABONO_A. Si trae una CLABE, el abono hacia esa cuenta falla a propósito con un 503. Está en application.yml (mexibanco.demo.fallar-abono-a) y en docker-compose.yml, y con valor vacío no hace nada. Se usa así:

```
FALLAR_ABONO_A=$DESTINO docker compose up -d transferencia
```

5. **Pruebas.**

Cambié el cuarto test, que antes comprobaba que nadie devolvía el cargo, y agregué dos: uno para que se devuelva el dinero y otro para que quede COMPENSACION_PENDIENTE.

La compensación funcionando [9:15 PM]

Prueba 1: Una transferencia normal:

Primero comprobé que el camino normal sigue funcionando. Mandé una transferencia de 100: respondió 201 con estado COMPLETADA (id 3), y los saldos quedaron en 700 el origen y 600 el destino.



Ilustración 7. Transferencia de 100 y consulta de las dos cuentas



Ilustración 8. Consulta de la cuenta origen: saldo 700

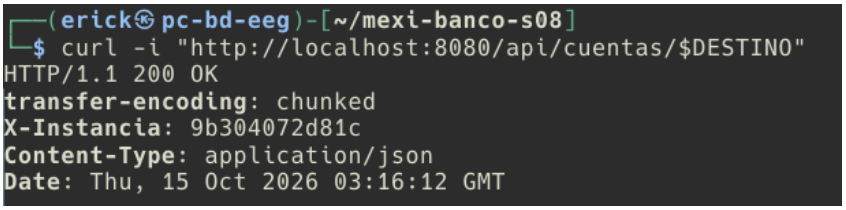


Ilustración 9. Consulta posterior de la cuenta destino

Prueba 2: Falla provocada del abono

Levanté transferencia con FALLAR_ABONO_A con la cuenta destino y mandé una transferencia de 50 por el gateway.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08]
└─$ FALLAR_ABONO_A=$DESTINO docker compose up -d transferencia
docker compose ps
[+] Running 3/3
 ✔ Container mexi-banco-v06-discovery-1      Healthy                                                    3.8s
 ✔ Container mexi-banco-v06-db-1             Healthy                                                    3.8s
 ✔ Container mexi-banco-v06-transferencia-1  Started                                                    4.1s
NAME                              IMAGE                          COMMAND                  SERVICE         CREATED          STATUS
PORTS
mexi-banco-v06-cuenta-1           mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          2 hours ago      Up 2 hours (healthy)
8080/tcp
mexi-banco-v06-cuenta-2           mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          2 hours ago      Up 2 hours (healthy)
8080/tcp
mexi-banco-v06-cuenta-3           mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          2 hours ago      Up 2 hours (healthy)
8080/tcp
mexi-banco-v06-db-1               postgres:17-alpine             "docker-entrypoint.s…"   db              2 hours ago      Up 2 hours (healthy)
5432/tcp
mexi-banco-v06-discovery-1        mexi-banco-v06-discovery       "java -jar app.jar"      discovery       2 hours ago      Up 2 hours (healthy)
0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
mexi-banco-v06-gateway-1          mexi-banco-v06-gateway         "java -jar app.jar"      gateway         2 hours ago      Up 2 hours (healthy)
0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-v06-movimiento-1       mexi-banco-v06-movimiento      "java -jar app.jar"      movimiento      2 hours ago      Up 2 hours (healthy)
8080/tcp
mexi-banco-v06-notificacion-1     mexi-banco-v06-notificacion    "java -jar app.jar"      notificacion    2 hours ago      Up 2 hours (healthy)
8080/tcp
mexi-banco-v06-spei-1             mexi-banco-v06-spei            "java -jar app.jar"      spei            2 hours ago      Up 2 hours (healthy)
8080/tcp
mexi-banco-v06-transferencia-1    mexi-banco-v06-transferencia   "java -jar app.jar"      transferencia   4 seconds ago    Up Less than a second (health: starting)
8080/tcp
```

Ilustración 10. Transferencia levantada con FALLAR_ABONO_A=$DESTINO.

La respuesta fue 409 con el mensaje de que la transferencia 4 quedó en estado COMPENSADA: el abono falló y el dinero se devolvió a la cuenta de origen. Los saldos antes y después fueron los mismos:

Momento	**Saldo origen**	**Saldo destino**
Antes de la transferencia fallida	700	600
Después (compensada)	700	600

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08]
└─$ curl -i "$URL_O"
curl -i -X POST http://localhost:8080/api/transferencias \
  -H 'Content-Type: application/json' \
  -d "{\"claveOrigen\":\"$ORIGEN\",\"claveDestino\":\"$DESTINO\",\"monto\":50}"
curl -i "$URL_O"
curl -i "$URL_D"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: d1694f575c78
Content-Type: application/json
Date: Thu, 15 Oct 2026 03:19:31 GMT

{"id":6,"clabe":"S08O1792027643","titular":"Origen","saldo":700.00}HTTP/1.1 409 Conflict
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Thu, 15 Oct 2026 03:19:31 GMT

{"type":"about:blank","title":"Conflict","status":409,"detail":"Transferencia 4 en estado COMPENSADA: el abono falló y el dinero se devolvió a la cuenta de origen."
,"instance":"/transferencias"}HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 9b304072d81c
Content-Type: application/json
Date: Thu, 15 Oct 2026 03:19:31 GMT

{"id":6,"clabe":"S08O1792027643","titular":"Origen","saldo":700.00}HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: d2fcc81bbcd0
Content-Type: application/json
Date: Thu, 15 Oct 2026 03:19:31 GMT
```

Ilustración 11. Saldo del origen antes (700.00), respuesta 409 Conflict con COMPENSADA, y saldos después (origen 700.00, destino 600.00)

Estado final en la base. La transferencia 4 quedó COMPENSADA, con el detalle "Falló el abono (Falla provocada para la prueba) se devolvió el dinero". En la tabla de movimientos quedó el registro de la devolución con la referencia "compensación de la transferencia #4".

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08]
└─$ docker compose exec db psql -U mexibanco -d transferencia -c "select id, estado, detalle from transferencia order by id;"
 id |   estado    |                                detalle
----+-------------+---------------------------------------------------------------------
  1 |             |
  2 |             |
  3 | COMPLETADA  |
  4 | COMPENSADA  | Falló el abono (Falla provocada para la prueba); se devolvió el dinero
(4 rows)


┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08]
└─$ docker compose exec db psql -U mexibanco -d movimiento -c "select clabe_cuenta, tipo, monto, referencia from movimiento order by id desc limit 3;"
  clabe_cuenta   |          tipo          |  monto  |             referencia
-----------------+------------------------+---------+-------------------------------------
 S0801792027643 | TRANSFERENCIA_RECIBIDA |   50.00 | compensación de la transferencia #4
 S08D1792027715 | TRANSFERENCIA_RECIBIDA |  100.00 | de S0801792027643
 S0801792027643 | TRANSFERENCIA_ENVIADA  | -100.00 | a S08D1792027715
(3 rows)
```

Ilustración 12. Tabla de transferencias (ids 1 y 2 de la práctica anterior, 3 COMPLETADA, 4 COMPENSADA) y los últimos movimientos: abono de 50 con la referencia de compensación.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08]
└─$ docker compose exec db psql -U mexibanco -d movimiento -c "select clabe_cuenta, tipo, monto, referencia from movimiento order by id desc limit 5;"
  clabe_cuenta   |          tipo          |  monto  |             referencia
-----------------+------------------------+---------+-------------------------------------
 S08D1792027715 | TRANSFERENCIA_RECIBIDA |   50.00 | de S0801792027643
 S0801792027643 | TRANSFERENCIA_ENVIADA  |  -50.00 | a S08D1792027715
 S0801792027643 | TRANSFERENCIA_RECIBIDA |   50.00 | compensación de la transferencia #4
 S08D1792027715 | TRANSFERENCIA_RECIBIDA |  100.00 | de S0801792027643
 S0801792027643 | TRANSFERENCIA_ENVIADA  | -100.00 | a S08D1792027715
(5 rows)
```

Ilustración 13. Aparece "compensación de la transferencia #4" y arriba, una transferencia posterior de 50 (la del primer intento de la Parte 5).

Por qué funciona. Al fallar el último paso, la saga no se queda esperando ni responde un error sin más: ejecuta la compensación del paso anterior, que es devolver el cargo.

Prueba 3: La compensación también falla

Repetí la prueba de la Parte 0, ahora con la saga. Puse una pausa de 10 segundos entre el cargo y el abono (para que el gateway, que espera 15 s, sí alcanzara a responder), mandé una transferencia de 50 y apagué cuenta durante la pausa.

Primer intento. No alcancé a apagar cuenta dentro de los 10 segundos, así que esa transferencia terminó COMPLETADA (es la fila 5 de la tabla de estados). Por eso, antes de repetir la prueba, los saldos eran 650 y 650.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08]
└─$ curl -i "$URL_O"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: d2fcc81bbcd0
Content-Type: application/json
Date: Thu, 15 Oct 2026 03:29:01 GMT

{"id":6,"clabe":"S0801792027643","titular":"Origen","saldo":650.00}
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08]
└─$ curl -i "$URL_D"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: d2fcc81bbcd0
Content-Type: application/json
Date: Thu, 15 Oct 2026 03:29:07 GMT

{"id":7,"clabe":"S08D1792027715","titular":"Destino","saldo":650.00}
```

Ilustración 14. Saldos antes de repetir la prueba: origen 650.00, destino 650.00

Segundo intento. Apagué cuenta con docker compose stop cuenta y mandé la transferencia. Respondió 409 con el mensaje de COMPENSACION_PENDIENTE: el dinero salió del origen y la devolución quedó pendiente.

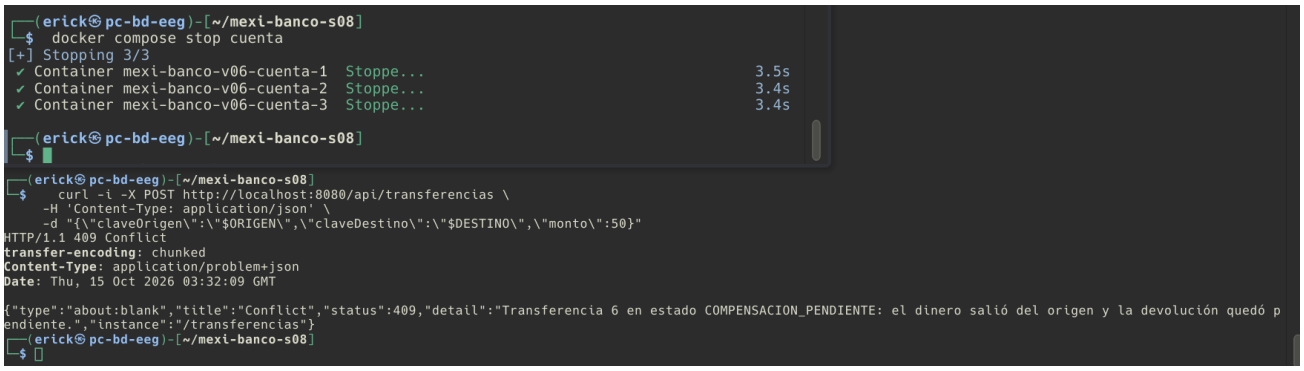


Ilustración 15. Cuenta apagada (3 copias detenidas) y la respuesta 409 Conflict de la transferencia 6 en estado COMPENSACION_PENDIENTE

En la base quedó así:

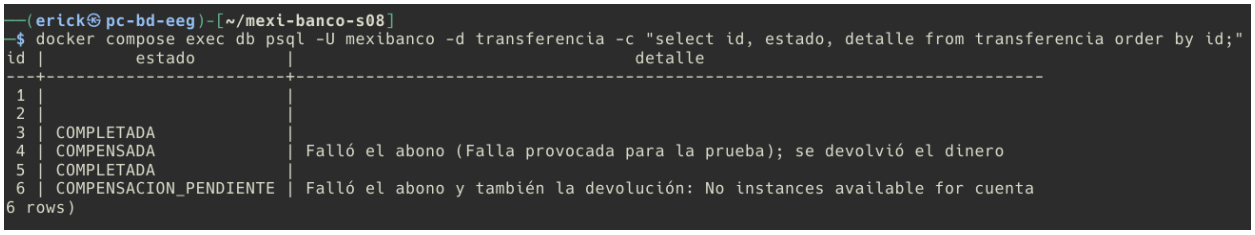


Ilustración 16. Tabla de transferencias: la fila 6 quedó en COMPENSACION_PENDIENTE, con el detalle "No instances available for cuenta".

El detalle dice que el balanceador no encontró ninguna copia viva de cuenta, ni para el abono ni para la devolución. El saldo del origen quedó en 600, que son los 650 de antes menos los 50 que salieron y no regresaron:

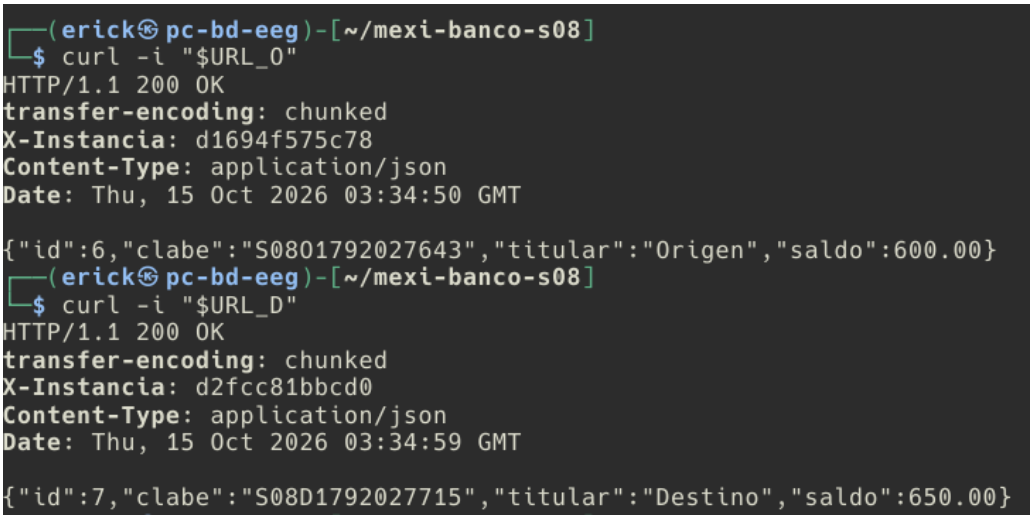


Ilustración 17. Saldos después: origen 600.00 (el dinero sigue fuera) y destino 650.00

Qué cambió respecto a la Parte 0. Antes, el dinero se perdía y no quedaba ninguna señal de lo que había pasado. Ahora la transferencia no se queda en `CARGADA` para siempre: queda en un estado conocido que dice exactamente qué falta, que es terminar la devolución. Es el mismo principio de la lectura de Outbox: guardar el estado en la base local para que, si algo se cae, se sepa en qué punto se quedó.

Cómo la terminaría. Con un proceso programado (por ejemplo, cada minuto) que busque las transferencias en `COMPENSACION_PENDIENTE` y reintente la devolución a la cuenta origen. Cuando cuenta vuelva a estar disponible, el reintento funciona y la transferencia pasa a `COMPENSADA`. Si sigue fallando, se reintenta con esperas cada vez más largas (1 s, 2 s, 4 s...) y, pasado cierto número de intentos, se avisa a una persona.

Por qué la compensación debe ser idempotente. Porque el reintento puede ejecutarse más de una vez, por ejemplo si la devolución sí llegó a cuenta pero la respuesta se perdió. Si no fuera idempotente, el cliente recibiría su dinero dos veces. Mi código lo evita con el estado: solo devuelve el dinero si la transferencia está en `CARGADA` o `COMPENSACION_PENDIENTE`, y marca `COMPENSADA` justo después de que la devolución funciona.

Una limitación que encontré. El servicio cuenta no guarda ninguna llave de idempotencia en sus operaciones de cargo y abono, y en esta práctica solo entrego transferencia. Por eso la protección está en el estado de mi saga y no en cuenta. Si el abono diera timeout, no sabría si se aplicó o no, y devolver el dinero podría duplicarlo. Lo correcto sería que cuenta aceptara una llave única por operación.

El circuit breaker

Puse un circuit breaker en las llamadas de transferencia a cuenta. Lo hice a mano, siguiendo la lámina 37: una clase CircuitBreaker con tres estados, un contador de fallas seguidas y un tiempo de espera. Lo uso en CuentaCliente, así que consultar, cargar y abonar comparten el mismo circuito. Usé los valores de la lámina: 5 fallas seguidas para abrir y 30 segundos de espera antes de probar.

```
public <T> T llamar(Supplier<T> llamada) {
    permitir();                        // si está ABIERTO, falla sin llamar
    try {
        T resultado = llamada.get();
        exito();                       // cierra el circuito y pone las fallas en 0
        return resultado;
    } catch (ErrorRemotoException e) {
        if (e.getEstado() < 500) { exito(); } else { falla(); }   // un 404 o 422 no
es una falla de cuenta
        throw e;
    } catch (RuntimeException e) {
        falla();                       // sin conexión, timeout, "No instances
available"
        throw new ErrorRemotoException(nombre, 503, "No se pudo contactar a " +
nombre + ": " + e.getMessage());
    }
}
```

Una decisión mía fue que solo cuentan como falla los casos en que cuenta no responde (sin conexión, timeout o un 5xx). Un 404 o un 422 (saldo insuficiente) significan que cuenta sí contestó, aunque haya dicho que no. Si no, alguien pidiendo cuentas que no existen podría abrir el circuito sin que cuenta esté caída. Además, en el estado medio abierto solo dejo pasar una llamada de prueba.

El timeout ya venía en el proyecto: `ClientesHttp` limita la conexión a 2 s y la lectura a 5 s, así que ninguna llamada espera para siempre.

Los tres estados y cómo cambian

Estado	Qué hace	Cambia a...
CERRADO	Las llamadas pasan y se cuentan las fallas seguidas.	ABIERTO al llegar a 5 fallas seguidas.
ABIERTO	No llama a cuenta: falla al instante con un 503.	MEDIO_ABIERTO cuando pasan 30 s.
MEDIO_ABIERTO	Deja pasar **una** llamada de prueba.	CERRADO si la prueba sale bien, ABIERTO si falla.

Prueba 1: el circuito se abre

Estado CERRADO. Con cuenta encendida, una transferencia normal de 10 pasa sin problema: 201 con estado COMPLETADA. El circuito está cerrado y no hay mensajes en el log.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08]
└─$ curl -i -X POST http://localhost:8080/api/transferencias \
  -H 'Content-Type: application/json' \
  -d "{\"claveOrigen\":\"$ORIGEN\",\"claveDestino\":\"$DESTINO\",\"monto\":10}"
HTTP/1.1 201 Created
Transfer-encoding: chunked
Content-Type: application/json
Date: Thu, 15 Oct 2026 04:20:51 GMT

{"id":7,"claveOrigen":"S08O1792027643","claveDestino":"S08D1792027715","monto":10,"fecha":"2026-10-15T04:20:40.963137050Z","estado":"COMPLETADA"
,"detalle":null}
```

Ilustración 18. Transferencia de 10 con el circuito cerrado: 201 Created, COMPLETADA (id 7).

El circuito se abre. Apagué cuenta con docker compose stop cuenta y mandé 8 transferencias seguidas. En las peticiones 1 a 5 el servicio sí intentó llamar a cuenta y falló (503, "No se pudo contactar a cuenta"). A partir de la sexta, el circuito ya estaba abierto y respondió 503 "Circuito abierto" sin llamar a cuenta.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08/transferencia]
└─$ docker compose stop cuenta
[+] Stopping 3/3
 ✔ Container mexi-banco-v06-cuenta-3  Stopped                                                                    3.5s
 ✔ Container mexi-banco-v06-cuenta-1  Stopped                                                                    3.4s
 ✔ Container mexi-banco-v06-cuenta-2  Stopped                                                                    3.4s

┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08/transferencia]
└─$ for i in {1..8}; do
  echo "$i  $(curl -s -X POST http://localhost:8080/api/transferencias \
    -H 'Content-Type: application/json' \
    -d "{\"claveOrigen\":\"$ORIGEN\",\"claveDestino\":\"$DESTINO\",\"monto\":10}" | grep -o '"status":[0-9]*,\?\|"detail":"[^"]*"' | tr '\n' ' '
)"
done
1  "status":503, "detail":"No se pudo contactar a cuenta: No instances available for cuenta"
2  "status":503, "detail":"No se pudo contactar a cuenta: No instances available for cuenta"
3  "status":503, "detail":"No se pudo contactar a cuenta: No instances available for cuenta"
4  "status":503, "detail":"No se pudo contactar a cuenta: No instances available for cuenta"
5  "status":503, "detail":"No se pudo contactar a cuenta: No instances available for cuenta"
6  "status":503, "detail":"Circuito abierto: cuenta no está disponible, se rechaza sin llamarlo"
7  "status":503, "detail":"Circuito abierto: cuenta no está disponible, se rechaza sin llamarlo"
8  "status":503, "detail":"Circuito abierto: cuenta no está disponible, se rechaza sin llamarlo"
```

Ilustración 19. Cuenta apagada y las 8 peticiones: 1 a 5 con "No se pudo contactar a cuenta", 6 a 8 con "Circuito abierto: ... se rechaza sin llamarlo".

Las primeras cinco no fueron lentas porque docker compose stop da de baja a cuenta del directorio al instante, así que la diferencia está en el mensaje y no en el tiempo. En el log quedó la apertura del circuito:

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08/transferencia]
└─$ docker compose logs transferencia | grep Circuito
transferencia-1  | 2026-10-15T04:45:06.958Z  INFO 1 --- [transferencia] [nio-8080-exec-5] mx.mexibanco.clientes.CircuitBreaker     : Circuito de cuenta: CERRADO -> ABIERTO
transferencia-1  | 2026-10-15T04:46:21.127Z  INFO 1 --- [transferencia] [nio-8080-exec-4] mx.mexibanco.clientes.CircuitBreaker     : Circuito de cuenta: ABIERTO -> MEDIO_ABIERTO
transferencia-1  | 2026-10-15T04:46:21.130Z  INFO 1 --- [transferencia] [nio-8080-exec-4] mx.mexibanco.clientes.CircuitBreaker     : Circuito de cuenta: MEDIO_ABIERTO -> ABIERTO
```

Ilustración 20. Log de transferencia: CERRADO -> ABIERTO (04:45:06 GMT) y, pasados los 30 s, ABIERTO -> MEDIO_ABIERTO -> ABIERTO

Prueba 2: La recuperación

Pasados los 30 s, el circuito pasó a medio abierto y dejó pasar una llamada de prueba. Como cuenta seguía apagada, la prueba falló y el circuito volvió a abrirse.

Después encendí cuenta con docker compose start cuenta y esperé a que las tres copias estuvieran healthy. Mandé una transferencia, que dio 201 COMPLETADA (id 8). Esta vez la llamada de prueba salió bien y el circuito se cerró. El log completo muestra todas las transiciones:

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08/transferencia]
└─$ docker compose ps
NAME                          IMAGE                          COMMAND                  SERVICE         CREATED          STATUS                    PORTS
mexi-banco-v06-cuenta-1       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          4 hours ago      Up 14 seconds (healthy)   8080/tcp
mexi-banco-v06-cuenta-2       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          4 hours ago      Up 14 seconds (healthy)   8080/tcp
mexi-banco-v06-cuenta-3       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          4 hours ago      Up 14 seconds (healthy)   8080/tcp
mexi-banco-v06-db-1           postgres:17-alpine             "docker-entrypoint.s…"   db              4 hours ago      Up 4 hours (healthy)      5432/tcp
mexi-banco-v06-discovery-1    mexi-banco-v06-discovery       "java -jar app.jar"      discovery       4 hours ago      Up 4 hours (healthy)      0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
mexi-banco-v06-gateway-1      mexi-banco-v06-gateway         "java -jar app.jar"      gateway         4 hours ago      Up 4 hours (healthy)      0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-v06-movimiento-1   mexi-banco-v06-movimiento      "java -jar app.jar"      movimiento      4 hours ago      Up 4 hours (healthy)      8080/tcp
mexi-banco-v06-notificacion-1 mexi-banco-v06-notificacion    "java -jar app.jar"      notificacion    4 hours ago      Up 4 hours (healthy)      8080/tcp
mexi-banco-v06-spei-1         mexi-banco-v06-spei            "java -jar app.jar"      spei            4 hours ago      Up 4 hours (healthy)      8080/tcp
mexi-banco-v06-transferencia-1 mexi-banco-v06-transferencia  "java -jar app.jar"      transferencia   5 minutes ago    Up 5 minutes (healthy)    8080/tcp

┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08/transferencia]
└─$ curl -i -X POST http://localhost:8080/api/transferencias \
  -H 'Content-Type: application/json' \
  -d "{\"claveOrigen\":\"$ORIGEN\",\"claveDestino\":\"$DESTINO\",\"monto\":10}"
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Thu, 15 Oct 2026 04:49:41 GMT

{"id":8,"claveOrigen":"S08D1792027643","claveDestino":"S08D1792027715","monto":10,"fecha":"2026-10-15T04:49:40.985452517Z","estado":"COMPLETADA","detalle":null}
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s08/transferencia]
└─$ docker compose logs transferencia | grep Circuito
transferencia-1  | 2026-10-15T04:45:06.958Z  INFO 1 --- [transferencia] [nio-8080-exec-5] mx.mexibanco.clientes.CircuitBreaker     : Circuito de cuenta: CERRADO -> ABIERTO
transferencia-1  | 2026-10-15T04:46:21.127Z  INFO 1 --- [transferencia] [nio-8080-exec-4] mx.mexibanco.clientes.CircuitBreaker     : Circuito de cuenta: ABIERTO -> MEDIO_ABIERTO
transferencia-1  | 2026-10-15T04:46:21.130Z  INFO 1 --- [transferencia] [nio-8080-exec-4] mx.mexibanco.clientes.CircuitBreaker     : Circuito de cuenta: MEDIO_ABIERTO -> ABIERTO
transferencia-1  | 2026-10-15T04:49:40.595Z  INFO 1 --- [transferencia] [nio-8080-exec-1] mx.mexibanco.clientes.CircuitBreaker     : Circuito de cuenta: ABIERTO -> MEDIO_ABIERTO
transferencia-1  | 2026-10-15T04:49:40.823Z  INFO 1 --- [transferencia] [nio-8080-exec-1] mx.mexibanco.clientes.CircuitBreaker     : Circuito de cuenta: MEDIO_ABIERTO -> CERRADO
```

Ilustración 21. Docker compose con las tres copias healthy, transferencia de 10 con 201 COMPLETADA, y el log completo con las cinco transiciones.

Transición	**Qué pasó**
CERRADO -> ABIERTO	Se acumularon 5 fallas seguidas con cuenta apagada.
ABIERTO -> MEDIO_ABIERTO	Pasaron los 30 s y se dejó pasar una llamada de prueba.
MEDIO_ABIERTO -> ABIERTO	La prueba falló porque cuenta seguía apagada, y el circuito se volvió a abrir.
ABIERTO -> MEDIO_ABIERTO	Cuenta ya estaba encendida, se hizo una nueva prueba.
MEDIO_ABIERTO -> CERRADO	La prueba salió bien (201 COMPLETADA) y el circuito se cerró.

Por último, el tablero de Eureka con cuenta de nuevo registrada con sus tres copias:

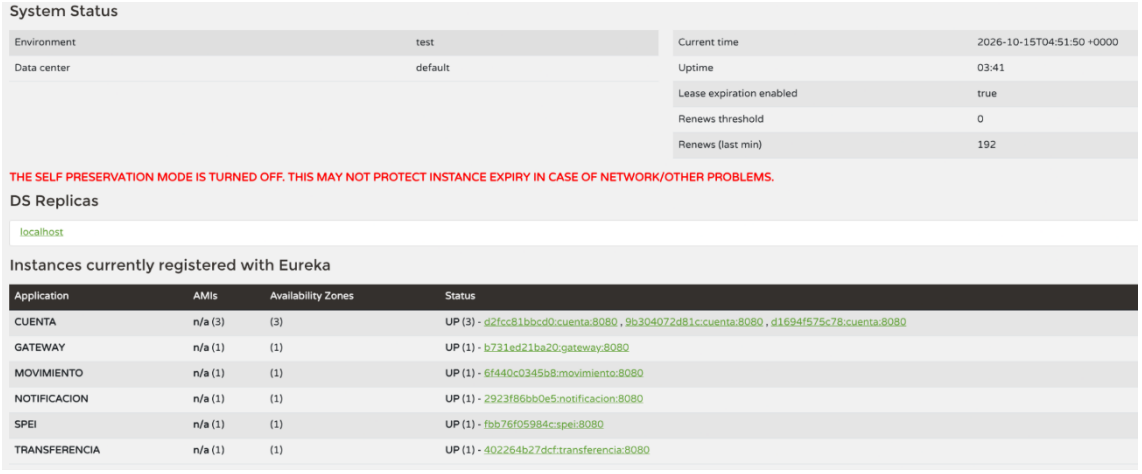


Ilustración 22. Eureka con CUENTA en UP(3)

Cómo se combina con la saga

Con el circuito abierto, si una transferencia ya hizo el cargo y el abono es rechazado, la devolución también se rechaza al instante y la transferencia queda en `COMPENSACION_PENDIENTE`. Es lo mismo que en la Parte 1, pero sin quedarse colgada esperando. Y si el circuito ya está abierto al inicio, ni siquiera se empieza, la consulta de las cuentas falla antes de guardar o cobrar nada.

Lo que la saga no te da: aislamiento

Una saga da atomicidad eventual, consistencia y durabilidad, pero no da aislamiento. Cada paso se confirma por separado y es visible para todos apenas se guarda, así que otros pueden ver los estados intermedios.

Alguien que consulta el saldo de la cuenta origen entre el cargo y la compensación ve un saldo menor al real: el dinero ya salió, pero todavía no ha regresado. Eso lo vi en mi prueba: con la transferencia en COMPENSACION_PENDIENTE, el origen quedó en 600 en vez de 650 y se quedó así hasta que alguien terminé la devolución. Es parecido a una lectura sucia, porque se ve un dato que luego se deshace.

Hay consecuencias reales, ya que otra operación que dependa de ese saldo (por ejemplo, otra transferencia desde el mismo origen) puede rechazarse por "saldo insuficiente" aunque el dinero vaya a volver.

Mi saga lo reduce, pero no lo elimina. La tabla de transferencias muestra los estados CARGADA y COMPENSACION_PENDIENTE, que marcan lo que todavía no es definitivo. La compensación solo se hace si el estado lo permite, para no devolver dos veces. Y los avisos van al final porque no se pueden deshacer. Lo que no puedo

evitar es que alguien consulte cuenta en ese intervalo y vea el saldo intermedio, porque cuenta no sabe nada de la saga.

Esta es la diferencia con 2PC, ahí habría candados hasta el COMMIT y nadie vería el estado intermedio, pero con el costo de bloquear y de depender del coordinador. En una saga se elige disponibilidad y se acepta mostrar estados intermedios.

Cierre

El monolito tenía un `@Transactional` que daba todo o nada. Al partirlo en servicios, esa garantía pasó a mi diseño: la saga guarda el estado en cada paso y compensa si algo falla, y el circuit breaker evita que la transferencia se quede colgada esperando a un servicio caído. Lo que no se resuelve solo es el aislamiento y la compensación que también falla, y para eso necesito estados pendientes e idempotencia.